PARTE 1 DE 4 · 3 MINUTOS

Santiago Arango

Introducción, tecnologías y arquitectura

Abres la exposición. Explicas **qué hace el sistema**, **con qué está construido** y **cómo está organizado el código**. No entres en detalle de cada clase: tú das el mapa general para que lo que viene después tenga sentido.

Orden de la exposición

#	Quién	Tema	Tiempo
1	Santiago Arango	Introducción, tecnologías y arquitectura	3 min
2	Alex Monroy	Modelo de datos: entidades, relaciones y repositorios	3 min
3	Jose Vasquez	Lógica de negocio: servicios, reglas y excepciones	3-4 min
4	Isaac	API REST, validaciones, errores y demo en vivo	4-5 min

Antes de empezar

- IntelliJ abierto con el árbol de paquetes desplegado hasta **co.Proyecto_Backendll**.
- Pestañas abiertas: **pom.xml** y **application.yml**.
- XAMPP con MySQL encendido y la aplicación ya corriendo, para que la demo no tenga que esperar.

Guion paso a paso

Cada paso dice qué mostrar en pantalla y qué decir. No lo leas palabra por palabra: úsalo como guía y dilo con tus palabras.

1. Presentación del grupo y del proyecto

QUÉ DECIR

Buenos días. Somos Santiago Arango, Alex Monroy, Jose Vasquez e Isaac, y les vamos a presentar una API REST para gestionar estudiantes, cursos y matrículas. Permite registrar estudiantes, crear cursos con un cupo máximo y matricular estudiantes respetando reglas como no repetir correos, no pasarse del cupo y no matricular dos veces a la misma persona.

2. Tecnologías

Muestra: pom.xml, bajando por las dependencias.

Tecnología	Para qué la usamos
Java 21 + Spring Boot 4	Lenguaje y framework; Spring Boot trae el servidor Tomcat incluido.
Web MVC	Crear los endpoints REST.
Data JPA (Hibernate)	Guardar y consultar datos sin escribir SQL a mano.
Validation	Validar los datos que llegan (correo válido, campos obligatorios).
MySQL (XAMPP) + Maven	Base de datos y manejo de dependencias.

QUÉ DECIR

Está hecho en Java 21 con Spring Boot, y Maven maneja las librerías desde este pom.xml. Las principales son Web MVC para los endpoints, Data JPA para conectarnos a la base de datos y Validation para revisar los datos que nos envían. La base de datos es MySQL con XAMPP.

3. Configuración

Muestra: application.yml

```
server:
  port: 8888
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mi_base_de_datos
  jpa:
    hibernate:
      ddl-auto: update
```

QUÉ DECIR

La aplicación corre en el puerto 8888 y aquí está la conexión a MySQL. Lo importante es ddl-auto en update: Hibernate crea y actualiza las tablas a partir de nuestras clases, así que no escribimos el CREATE TABLE a mano.

4. Arquitectura y recorrido de una petición

Muestra: El árbol de paquetes. Señala cada carpeta mientras hablas.

```
POST /students
  -> controller   recibe la petición y valida el JSON
  -> service      aplica las reglas (correo no repetido)
  -> repository   guarda en MySQL
  -> dto          se responde en JSON con código 201
```

QUÉ DECIR

El proyecto está dividido en capas y cada una tiene una sola responsabilidad. Cuando creamos un estudiante, la petición llega al controlador, que valida los datos; pasa al servicio, que revisa las reglas; el repositorio lo guarda en MySQL, y se responde en JSON. Separarlo así hace el código más fácil de mantener: si cambiamos la base de datos, los controladores no se enteran.

- **controller / dto:** entrada y salida de la API (Isaac).

- **service:** reglas de negocio (Jose).
- **domain / repository:** entidades y acceso a la base de datos (Alex).
- **presentation/advice:** manejo de errores. **CorsConfig:** permite al frontend del puerto 5173.

Preguntas que te pueden hacer

¿Por qué separar en capas?

Cada capa tiene una responsabilidad. Si las reglas estuvieran en el controlador no se podrían reutilizar ni probar por separado.

¿Qué hace ddl-auto: update?

Al arrancar compara las entidades con las tablas y crea o agrega lo que falte, sin borrar datos.

¿Para qué es CorsConfig?

El navegador bloquea peticiones entre puertos distintos; el frontend corre en el 5173 y la API en el 8888.

Cierre y paso al siguiente

> **QUÉ DECIR**
> *Esa es la estructura general. Ahora Alex les explica cómo modelamos los datos.*

Consejos

- Eres el primero: habla despacio y con seguridad, marca el tono del grupo.
- Ensayen una vez los cuatro seguidos con cronómetro. Si alguien se pasa, el que sufre es la demo del final.
- Cuando muestren código, hagan zoom en IntelliJ (Ctrl + rueda del mouse) para que se lea desde atrás.
- Si no saben una respuesta, digan qué parte del código la resuelve o que lo revisan; es mejor que inventar.